# Are Self-Evolving Agent Skills Learning Skills?

**Anonymous submission**


## Abstract

We introduce a lifecycle audit for determining whether score gains from self-evolving agent skills represent retained skill improvement. A post-revision score alone is insufficient because optimizer variation, validation selection, execution randomness, and verifier choice can improve the endpoint without creating a durable artifact effect. We combine repeated evaluations, controlled task variations, and lifecycle interventions to examine whether improvements from skill evolution persist and transfer, and to trace their origins to feedback, revision, selection, skill content, and evaluation. The audit separates five evidential claims: selected gain, retained artifact effect, bounded transfer, evolution-pathway dependence, and measurement dependence. This claim-grounded framework lets self-evolving systems report what their experiments establish without equating a better benchmark score with reusable skill learning.


## Introduction

Persistent agent skills offer an appealing route to continual improvement. Instead of updating model parameters, an agent can revise an external artifact that stores reusable instructions, scripts, references, templates, or other supporting resources. Recent systems generate, repair, select, and retain such artifacts from execution traces and evaluator feedback, often reporting higher post-evolution benchmark scores as evidence that the agent has learned a better skill (Li and Hu 2026; Liu et al. 2026b; Yang et al. 2026; Xia et al. 2026).

A higher score is not, by itself, an identified skill effect. The selected run may be unusually favorable; the optimizer may produce variable candidates; a validation gate may retain a narrow patch; repeated execution may eventually succeed without a durable behavioral change; and a verifier may reward only part of the intended behavior. These possibilities do not imply that self-evolution is ineffective. They imply that an endpoint score alone cannot locate where the improvement resides.

We therefore ask two connected questions. When self-evolution raises the benchmark score, does the revised skill itself retain a behavioral improvement under fresh deployment and controlled task variation? Regardless of whether it does, which parts of the evolution-and-evaluation lifecycle produced the observed gain?

We organize the evidence as a diagnosis of utility, behavioral scope, and gain attribution. Standard-task evaluations of SkillEvolver, SPARK, and CoEvoSkills first establish the observed utility of completed artifacts across two runs. Controlled task changes then reveal different scopes of usefulness, and artifact and trace inspection examines the decisions preserved during reuse. This motivates the next diagnostic step: locating an improvement in the artifact's evolution history. Complementary studies of SkillRevise, SkillHone, and SkillOpt use retention tests and lifecycle interventions to examine trace-conditioned revision, history-based iterative honing, and validation-gated editing. Each study addresses a specific part of the identification question.

Our audit follows the lifecycle from feedback to revision, selection or adoption, frozen artifact deployment, and measurement. We separate historical success from fresh retention, final-skill utility from actual parent-to-final revision effects, and same-task retention from controlled transfer. We then intervene on feedback, optimizer sampling, gate provenance, artifact components, and verifier choice. A short information-integrity check bounds direct answer or oracle exposure as an alternative explanation; it is not the paper's organizing hypothesis.

The resulting picture is deliberately more qualified than either *self-evolution works* or *self-evolution only exploits the evaluator*. Revised artifacts can cause real behavioral improvements, but the demonstrated effects are often narrow. Single-run or selected gains need not survive fresh deployment. Useful traces and feedback must be converted into an effective edit, adopted into the persistent artifact, and retained behaviorally. Finally, a better score can reflect compatibility with a measurement surface rather than broader task behavior.

Our central contribution is a claim-grounded lifecycle audit: each reported gain is matched to the intervention required to support it and to the strongest conclusion that intervention permits. We make three contributions:

- **Identification framework.** We separate a reported gain from retention, behavioral scope, evolution-and-selection pathway, and measurement dependence.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

# 自进化代理的技能是否在学习技能?

**匿名提交**


## 摘要

我们介绍了一种生命周期审计方法，用于判断自进化代理技能带来的分数提升是否代表持久的技能改进。仅凭修订后的分数是不够的，因为优化器变化、验证选择、执行随机性和验证器选择都可能在不创建持久性效果的情况下提高终点分数。我们结合重复评估、受控任务变化和生命周期干预，来检验技能进化带来的改进是否持久且可迁移，并追溯到反馈、修订、选择、技能内容和评估的来源。该审计将证据声明分为五类：选择增益、持久性效果、有界迁移、进化路径依赖和测量依赖。这个基于声明的框架允许自进化系统报告其实验结果，而不将更好的基准分数等同于可重用的技能学习。


## 引言

持久的代理技能为持续改进提供了一条有吸引力的途径。代理无需更新模型参数，而是可以修订存储可重用指令、脚本、参考资料、模板或其他支持资源的的外部工件。最近的系统从执行轨迹和评估器反馈中生成、修复、选择和保留此类工件，通常报告更高的进化后基准分数，以此作为代理学习了更好技能的证据（Li和Hu 2026；Liu等人 2026b；Yang等人 2026；Xia等人 2026）。

更高的分数本身并不能证明是某种已识别的技能效果。选定的运行可能特别有利；优化器可能产生不同的候选方案；验证门可能保留一个狭窄的补丁；重复执行最终可能在没有持久行为变化的情况下成功；并且验证器可能只奖励预期行为的一部分。这些可能性并不意味着自我进化是无效的。它们意味着仅凭终点分数无法定位改进所在的位置。

因此，我们提出两个相关的问题。当自我进化提高基准分数时，修订的技能它本身在新的部署和控制任务变化下能否保持行为改进？无论它是否能够做到，进化与评估生命周期中的哪些部分产生了观察到的收益？

我们将证据组织为效用、行为范围和收益归因的诊断。SkillEvolver、SPARK和CoEvoSkills的标准任务评估首先确定了两个运行中完成工件的可观察效用。受控任务的变化随后揭示了不同有用的范围，而工件和轨迹检查则考察了在重用过程中保留的决策。这促使了下一步的诊断步骤：定位工件进化历史中的改进。SkillRevise、SkillHone和SkillOpt的补充研究使用保留测试和生命周期干预来检查轨迹条件下的修订、基于历史的迭代精炼和验证门控编辑。每个研究都针对识别问题的特定部分。

我们的审计遵循从反馈到修订、选择或采纳、冻结工件部署以及测量的生命周期。我们将历史成功与新鲜留存、最终技能效用与实际父级到最终修订效果、相同任务留存与受控迁移区分开来。然后我们对反馈、优化器采样、门控溯源、工件组件和验证器选择进行干预。一个信息完整性检查将直接答案或或然器暴露作为替代解释进行边界；它不是论文的组织假设。

最终的结果故意比 自我进化有效 或 自我进化仅利用评估者 更有条件。修订的工件可以导致真正的行为改进，但展示的效果往往是狭隘的。单次运行或选定的收益不必在新部署中存活。有用的轨迹和反馈必须转化为有效的编辑、采纳到持久的工件中，并行为上保留。最后，更好的分数可以反映与测量表面的兼容性，而不是更广泛的任务行为。

我们的核心贡献是一个基于主张的生命周期审计：每个报告的收益都与支持它的干预措施以及干预措施允许的最强结论相匹配。我们做出了三项贡献：

- **识别框架**。 我们将报告的增益与保持、行为范围、进化与选择路径以及测量依赖性分开。

这是一个仅用于审阅目的的匿名提交。严禁分发、引用或公开分享本稿件。如果被接受，版权和出版细节将出现在最终版本中。

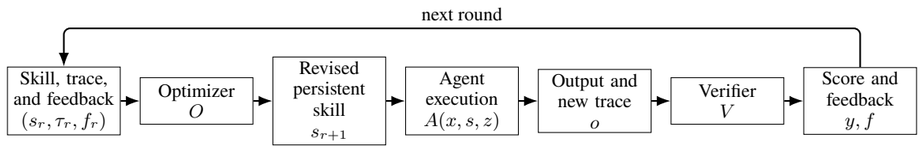



Figure 1: Minimal self-evolution lifecycle. The update operator includes any method-native validation, selection, merge, or adoption before the revised skill is retained.

- **Lifecycle audit.** We apply matched interventions across three representative skill-evolution mechanisms without turning heterogeneous studies into a method leaderboard.
- **Reporting protocol.** We specify the lineage, contrasts, and artifacts required to support each claim level, from selected gain to bounded transfer.

## Problem Formulation

### Skill Revision and Evaluation

At revision round $r$, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where $s_r$ is the current skill, $\tau_r$ is an execution trace, and $f_r$ is feedback that may include a verifier result, a gold answer, a diagnosis, or evaluator feedback. The operator $O$ denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not $s_{r+1}$.

The agent then executes task $x$ with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where $z_r$ represents model sampling, tool use, and environment state, and $o_r$ is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

### The Identification Question

Most evaluations observe:

$$y_{r+1} > y_r. \tag{4}$$

This inequality describes two evaluations; it does not identify what the persistent skill learned. The revised artifact may encode a retained behavioral improvement, the selected score may depend on optimizer or execution variation, or the output may better match the verifier's measurement surface. Infrastructure-invalid observations, such as a verifier that never reaches its semantic assertions, are neither successes nor failure mechanisms and must be repaired or excluded.

Accordingly, we separate two questions:

1. **Effect existence and scope:** Does the revised skill retain a behavioral improvement under fresh deployment, and how far does that effect extend under controlled task variation?
2. **Effect attribution:** Which feedback, revision, selection, artifact, execution, and measurement mechanisms produced the observed score change?

The following experiments address these questions in turn.

## Diagnosing the Scope of Skill Utility

The first diagnostic question is where a completed skill is useful. A positive source-task result can arise from content that applies across instances or from a procedure fitted to the original task. We distinguish these possibilities by first examining paired utility in two standard-task runs, then deploying frozen skills under three forms of task variation. Inspection of skill content and execution traces connects the resulting utility profiles to decisions made during deployment.

### Experimental Setup

We evaluate SkillEvolver (Zhang et al. 2026a), SPARK (Zhou et al. 2026b), and CoEvoSkills (Zhang et al. 2026b) on SkillsBench (Li et al. 2026), with two runs per method. Each run pairs an execution using the generated or evolved skill with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with $N$ tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left( r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{5}$$

We also count tasks with positive, negative, and zero differences.

### Source-Task Gains Recur across Runs

All three methods have positive mean paired gains in both runs (Table 1). SkillEvolver gains 25.61 and 25.02 points, CoEvoSkills gains 10.24 and 9.76, and SPARK gains 1.84 and 6.45. SkillEvolver and CoEvoSkills retain similar gains despite substantial changes in their no-skill scores. Their benefit is therefore more consistent across runs than their absolute skill-condition scores suggest.

The task-level results also differ. SkillEvolver has at least four times as many improvements as regressions in each run; CoEvoSkills improves 17 tasks in each run, with nine and ten regressions. SPARK's first run is nearly balanced, while its second has 13 improvements and five regressions. These results establish recurring source-task utility. We next ask whether the same frozen skills remain useful after the task changes.

### Task Changes Separate Different Scopes of Utility

We apply each source-task skill to controlled variants without further evolution. Input-representation changes (P1) al-

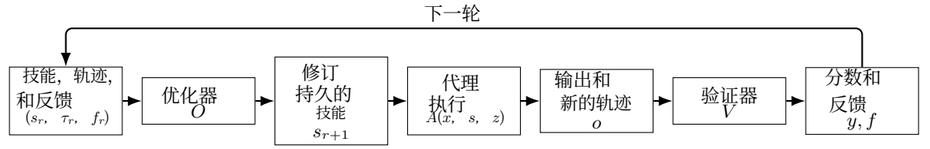



图1：最小自我进化生命周期。更新算子包括修订技能被保留之前的任何方法原生验证、选择、合并或采用。

- **生命周期审计。** 我们在三种代表性的技能进化机制上应用匹配干预，而不会将异质研究变成方法排行榜。
- **报告协议。** 我们指定支持从选定增益到有界迁移的每个主张级别的谱系、对比和工件。

## 问题公式化

### 技能修订与评估

在修订轮次 $r$ 中，一个优化器更新当前持久的技能工件：

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

其中 $s_r$ 是当前技能，$\tau_r$ 是一个执行轨迹，而 $f_r$ 是可能包含验证器结果、标准答案、诊断或评估器反馈的反馈。运算符 $O$ 表示方法完整的更新路径，包括其原生合并、验证、选择或采用机制。一个被拒绝或从未被采用的生成候选不是 $s_{r+1}$。

代理随后使用冻结的技能执行任务 $x$：

$$o_r = A(x, s_r, z_r), \tag{2}$$

其中 $z_r$ 表示模型采样、工具使用和环境状态，而 $o_r$ 是结果输出工件或状态变化。一个验证器将输出映射为一个分数：

$$y_r = V(x, o_r). \tag{3}$$

### 识别问题

大多数评估观察到：

$$y_{r+1} > y_r. \tag{4}$$

这个不等式描述了两种评估；它并不能识别持久技能学到了什么。修订后的工件可能编码了保留的行为改进，选定的分数可能取决于优化器或执行变化，或者输出可能更好地匹配验证器的测量表面。基础设施无效的观察，例如从未达到其语义断言的验证器，既不是成功机制也不是失败机制，必须被修复或排除。

因此，我们将问题分为两个：

1. **效果存在和范围：** 修订后的技能在全新部署下是否保留了行为改进，以及这种效果在受控任务变化下会延伸多远?
2. **效果归因：** 哪些反馈、修订、选择、工件、执行和测量机制导致了观察到的分数变化?

以下实验依次探讨这些问题。

## 诊断技能效用范围

第一个诊断问题是：一个完成的技能在何处有用。一个积极的源任务结果可能源于适用于多个实例的内容，也可能源于为原始任务拟合的程序。我们通过首先检查两个标准任务运行中的成对效用，然后以三种任务变化形式部署冻结的技能来区分这些可能性。通过检查技能内容和执行轨迹，可以将产生的效用特征与部署过程中做出的决策联系起来。

### 实验设置

我们在SkillsBench（Li等人，2026）上评估SkillEvolver（张等人，2026a）、SPARK（周等人，2026b）和CoEvoSkills（张等人，2026b），每种方法进行两次运行。每次运行将使用生成或进化的技能的执行与相同任务的技能无关的执行配对。每种方法的两次运行使用相同的任务集：SkillEvolver为81个任务，SPARK为84个任务，CoEvoSkills为85个任务。

我们报告在0-100量表上的平均验证器奖励，以及技能与技能无关条件之间的平均任务差异。对于包含 $N$ 个任务的运行，配对增益是

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left( r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{5}$$

我们还统计具有正、负和零差异的任务。

### 源任务增益跨运行重现

所有三种方法在两次运行中都具有正的平均配对增益（表1）。SkillEvolver获得了25.61和25.02分，CoEvoSkills获得了10.24和9.76分，SPARK获得了1.84和6.45分。尽管SkillEvolver和CoEvoSkills的无技能分数发生了巨大变化，但它们仍然保持了相似的增益。因此，它们在运行中的收益比其绝对技能条件分数所暗示的更为一致。

任务级别的结果也存在差异。SkillEvolver在每次运行中至少有四倍的改进多于回归；CoEvoSkills在每次运行中改进了17个任务，有九次和十次回归。SPARK的第一次运行几乎平衡，而第二次运行有13次改进和五次回归。这些结果确立了反复的源任务效用。我们接下来询问，在任务变化后，相同的冻结技能是否仍然有用。

### 任务变化分离不同的效用范围

我们将每个源任务技能应用于无进一步进化的控制变体。输入表示变化（P1）会

| Method | Run | Tasks | No skill | Skill | Δ | Improved | Regressed | Tied |
|---|---|---|---|---|---|---|---|---|
| SkillEvolver | 1 | 81 | 30.20 | 55.81 | +25.61 | 27 | 6 | 48 |
| SkillEvolver | 2 | 81 | 37.32 | 62.34 | +25.02 | 26 | 3 | 52 |
| SPARK | 1 | 84 | 17.82 | 19.66 | +1.84 | 9 | 8 | 67 |
| SPARK | 2 | 84 | 14.37 | 20.81 | +6.45 | 13 | 5 | 66 |
| CoEvoSkills | 1 | 85 | 42.21 | 52.44 | +10.24 | 17 | 9 | 59 |
| CoEvoSkills | 2 | 85 | 23.54 | 33.31 | +9.76 | 17 | 10 | 58 |

Table 1: Two runs per method on SkillsBench. Scores are mean verifier rewards multiplied by 100; Δ is the mean paired gain in percentage points. Each method uses an identical task set across its two runs.

**A. Overall utility on task variants**

| Method | Variants | No skill | Skill | Δ | Improved | Regressed | Tied |
|---|---|---|---|---|---|---|---|
| SkillEvolver | 222 | 41.87 | 53.81 | +11.94 | 47 | 20 | 155 |
| SPARK | 221 | 41.61 | 42.25 | +0.65 | 32 | 37 | 152 |
| CoEvoSkills | 221 | 41.61 | 36.60 | −5.01 | 26 | 41 | 154 |

**B. Utility by type of task change**

| Method | P1: Representation | | P2: Instance | | P3: Requirements | |
|---|---|---|---|---|---|---|
| | $N$ | Δ | $N$ | Δ | $N$ | Δ |
| SkillEvolver | 77 | +11.85 | 68 | +14.56 | 77 | +9.72 |
| SPARK | 77 | +2.00 | 68 | −5.23 | 76 | +4.53 |
| CoEvoSkills | 77 | −3.59 | 68 | −9.51 | 76 | −2.42 |

Table 2: Skill utility under task variation: descriptive summary of historical and recovery evaluations. Panel A reports overall outcomes; Panel B separates representation, instance, and requirement changes. Each method is paired with its own same-variant no-skill baseline. Scores and within-variant differences use the scale in Table 1.

ter names, column arrangements, or schemas; task-instance changes (P2) replace the data or instance structure; coupled-requirement changes (P3) alter or add requirements that must be satisfied together. These variations test whether a frozen skill remains useful when the executor must reinterpret inputs, recompute a solution, or accommodate new requirements.

The complete record contains 222 variants and 886 condition-level results across no skill and the three methods. Table 2 summarizes historical and recovery evaluations for 222 SkillEvolver variants and 221 each for SPARK and CoEvoSkills, pairing each skill condition with the no-skill result on the same variant.

**Source-task improvement does not imply useful generalization.** All three methods improve mean reward on the source tasks in both runs, but their variant results diverge. SkillEvolver retains a positive mean gain of 11.94 points, SPARK is close to neutral at +0.65, and CoEvoSkills falls below the no-skill baseline at −5.01. SkillEvolver also improves more variants than it regresses (47 versus 20), whereas SPARK and CoEvoSkills show the reverse. Thus, the source-task gains of the latter two methods do not translate into a consistent advantage on changed tasks.

**Replacing the instance produces the largest divergence.** SkillEvolver remains positive under all three changes, with its largest gain on P2 (+14.56). P2 is also where SPARK and CoEvoSkills decline most (−5.23 and −9.51). SPARK retains positive averages under representation and requirement changes, but loses its advantage when the underlying instance changes. SkillEvolver's positive results across P1–P3 indicate a broader observed scope of skill utility. CoEvoSkills' negative averages across all three changes show that its source-task benefit is particularly sensitive to reuse conditions.

## What the Evolved Skills Preserve

**Successful skills retain guidance for choosing and checking.** In exoplanet detection, the SkillEvolver skill supports successful detection after input columns are rearranged (P1) and after the task requires two signals (P3). The CoEvoSkills skill fails both variants: its use leads to stellar modulation being reported as the target signal, and a signal and its harmonic being treated as two separate signals. The SkillEvolver skill explicitly organizes candidate discovery, signal diagnosis, and refinement, including checks for misleading peaks. This case suggests that preserving criteria for accepting a result can be more useful under task changes than retaining a search routine alone. Both skills fail the replacement-instance P2 variant, so the successful checks address only part of this task family's variation.

A similar distinction appears in scheduling. The SkillEvolver and SPARK skills succeed on all three variants; the CoEvoSkills skill succeeds only on P3. In P1, using the CoEvoSkills skill yields an otherwise valid schedule that fails the requirement to place each operation at its earliest feasible time. The SkillEvolver skill explicitly includes this local-minimality requirement and baseline-relative repair rules. Here, successful reuse depends on preserving the task's repair conditions in the skill.

| 方法 | Run | 任务 | 无技能 | 技能 | Δ | 改进 | 退化 | Tied |
|---|---|---|---|---|---|---|---|---|
| SkillEvolver | 1 | 81 | 30.20 | 55.81 | +25.61 | 27 | 6 | 48 |
| SkillEvolver | 2 | 81 | 37.32 | 62.34 | +25.02 | 26 | 3 | 52 |
| SPARK | 1 | 84 | 17.82 | 19.66 | +1.84 | 9 | 8 | 67 |
| SPARK | 2 | 84 | 14.37 | 20.81 | +6.45 | 13 | 5 | 66 |
| CoEvoSkills | 1 | 85 | 42.21 | 52.44 | +10.24 | 17 | 9 | 59 |
| CoEvoSkills | 2 | 85 | 23.54 | 33.31 | +9.76 | 17 | 10 | 58 |

表1：SkillsBench上每个方法运行两次。分数是平均验证奖励乘以100； Δ 是平均配对增益（以百分比点为单位）。每种方法在其两次运行中使用相同的任务集。

**A. 在任务变体上的总体效用**

| 方法 | 变体 | 无技能 | 技能 | Δ | 改进 | 退化 | Tied |
|---|---|---|---|---|---|---|---|
| SkillEvolver | 222 | 41.87 | 53.81 | +11.94 | 47 | 20 | 155 |
| SPARK | 221 | 41.61 | 42.25 | +0.65 | 32 | 37 | 152 |
| CoEvoSkills | 221 | 41.61 | 36.60 | −5.01 | 26 | 41 | 154 |

**B. 按任务变化类型划分的效用**

| 方法 | P1：表示 | | P2：实例 | | P3：需求 | |
|---|---|---|---|---|---|---|
| | $N$ | Δ | $N$ | Δ | $N$ | Δ |
| SkillEvolver | 77 | +11.85 | 68 | +14.56 | 77 | +9.72 |
| SPARK | 77 | +2.00 | 68 | −5.23 | 76 | +4.53 |
| CoEvoSkills | 77 | −3.59 | 68 | −9.51 | 76 | −2.42 |

表 2：任务变化下的技能效用：历史和恢复评估的描述性摘要。A 面板报告总体结果；B 面板将表示、实例和需求变化分开。每种方法都与其自身的同变体无技能基线配对。分数和变体内差异使用表 1 中的量表。

这些变化测试冻结的技能在执行者必须重新解释输入、重新计算解决方案或适应新需求时是否仍然有用。

完整记录包含 222 个变体和 886 个条件级结果，涵盖无技能和三种方法。表2总结了 222 个 SkillEvolver 变体的历史和恢复评估，以及 SPARK 和 CoEvoSkills 各 221 个，将每个技能条件与同一变体上的无技能结果配对。

**源任务改进并不一定意味着有用的泛化**。 所有三种方法在两种运行中均提高了源任务的平均奖励，但它们的变体结果出现了分歧。SkillEvolver 保持了 11.94 点的正面平均增益，SPARK 在 +0.65 处接近中性，而 CoEvoSkills 低于无技能基线 −5.01。SkillEvolver 改进的变体比它退化的变体更多（47 对 20），而 SPARK 和 CoEvoSkills 则相反。因此，后两种方法的源任务增益并没有转化为在变更任务上的一致优势。

**替换实例产生了最大的发散**。SkillEvolver在所有三种变化下都保持正值，其在P2上的最大增益为+14.56。P2也是SPARK和CoEvoSkills下降最多的地方(−5.23 和 −9.51)。SPARK在表示和需求变化下仍保持正值平均，但当底层实例变化。SkillEvolver 在 P1– P3 上的积极结果表明观察到的技能效用范围更广。Co- EvoSkills 在所有三个变化上的负面平均值显示，其源任务收益对重用条件特别敏感。

## 进化技能保留什么

**成功的技能保留选择和检查的指导**。 在系外行星探测中，SkillEvolver技能在输入列重新排列（P1）后和任务需要两个信号（P3）后都支持成功探测。CoEvoSkills技能在这两种情况下都失败：其使用会导致恒星调制被报告为目标信号，并且一个信号及其谐波被视为两个独立的信号。SkillEvolver技能明确组织了候选发现、信号诊断和细化，包括对误导性峰值的检查。这种情况表明，在任务变化下，保留接受结果的标准的条件比保留搜索程序本身更有用。这两种技能都失败了replacement-instance P2变体，因此成功的检查仅解决了该任务家族变化的一部分。

在调度方面也存在类似的区别。SkillEvolver 和 SPARK 技能都能在三种变体上都取得成功；而 CoEvoSkills 技能仅在 P3 上成功。在 P1 中，使用 CoEvoSkills 技能会产生一个原本有效的调度，但会违反将每个操作放置在其最早可行时间的规则。SkillEvolver 技能明确包含了这一局部最优性要求以及相对于基线的修复规则。在这里，成功的重用取决于在技能中保留任务修复条件。

**The feedback used to build a skill helps explain its contents.** The two methods differ in how they turn experience into revisions. SkillEvolver compares successful and failed deployments of a candidate skill, then patches guidance that was missing or misleading. Its independent audit also checks for reliance on training-instance constants (Zhang et al. 2026a). These mechanisms directly target both omitted conditions and instance-specific assumptions. The rules retained in our scheduling and signal-detection skills are consistent with that emphasis: they describe what to check when reusing a procedure.

CoEvoSkills obtains detailed revision feedback from a co-evolving surrogate verifier; the ground-truth oracle returns only an opaque pass/fail signal (Zhang et al. 2026b). Consequently, discovering an omitted condition depends on the surrogate reconstructing it from the task and observed outputs. This offers a plausible explanation for a skill that covers many feasibility checks but misses a condition needed for successful reuse, as in the scheduling example. Both methods generate reusable skill packages; the relevant difference is which failure information becomes persistent guidance. Our interpretation is that feedback coverage and abstraction from the source instance help determine the scope of the resulting skill.

**Connecting utility to the evolution process.** The results identify useful skills and differences in their applicability; the examples suggest how retained guidance relates to those differences. The next studies test where improvements enter the lifecycle, using parent-to-final comparisons and interventions on feedback, selection, skill content, and verification in SkillRevise, SkillHone, and SkillOpt.

## Identifying Retained Skill Effects

### Evaluation Design and Contrasts

We analyze three persistent-skill systems with distinct revision mechanisms. SkillRevise converts execution traces and diagnoses into bounded repair principles (Liu et al. 2026b). SkillHone accumulates practice history and iteratively reflects on the persistent skill (Li and Hu 2026). SkillOpt produces structured bounded edits and uses a validation gate and rejected buffer before adoption (Yang et al. 2026). The systems are studied as representative mechanisms, not as directly comparable leaderboard entries.

At the paper level, the evidence is exploratory: individual interventions use frozen inputs and predeclared gates, but the combined studies are designed to identify candidate mechanisms rather than estimate benchmark-wide prevalence. A targeted 24-task SkillsBench panel tests whether historical outcomes survive fresh deployment. The panel is stratified to expose stability failures and method disagreements; it is not a random sample. Three controlled mechanism cases then compare complete parent and final artifacts under predeclared variations. Within each contrast, we hold the executor model, harness, task instance, budget, and verifier version fixed unless one is the named intervention; exact prompts, hashes, seeds, and stopping decisions are preserved in the artifact package. The task is the primary analysis unit, while repeated runs are deployments of the same task-condition pair.

Four comparisons must remain distinct. Historical-pass to fresh-repeat comparisons test retention of a selected outcome, not advantage over a baseline. Final-to-no-skill comparisons estimate fresh utility, not the effect of revision. Only actual-parent to final comparisons identify a revision effect. Applying that comparison to controlled variants tests bounded transfer, not population-average benefit.

### Retention under Fresh Deployment

This experiment tests whether historically selected outcomes survive when the final artifact is frozen and redeployed. Holding task, skill, and condition fixed does not make execution deterministic. Across the 24-task panel, 12 tasks change success status between their two repeats in at least one condition. Aggregate fresh success is 14/48 without a skill, 16/48 with the SkillRevise final artifact, and 12/48 with the SkillHone final artifact. Both matched-lift intervals cross zero (Table 3).

These results establish an identification failure in the common reporting practice: cached or selected success cannot be read as stable deployment capability. They do not establish that either revision method is ineffective. The panel is purposive, and it lacks each method's actual parent condition. Its proper conclusion is narrower: on this sample, neither final-versus-no-skill comparison establishes a clear average fresh-deployment advantage, and fresh repeats are necessary before a selected score is called retained.

### Revision Effects under Controlled Variations

These mechanism probes test whether the actual parent-to-final revision effect persists under predeclared task changes. In Invoice, the complete final skill repairs all 12 matched deployments and retains the effect on nine deployments drawn from three disjoint corpora that preserve the contract while changing the documents. This is strong evidence of a stable, same-contract revision effect, but not of broad cross-contract generalization.

The Energy case provides weaker but directionally consistent evidence: the complete final artifact improves the original setting and a 5% parameter variant in five of six deployments, compared with one of six for the parent. The sample is too small to support a prevalence or generality claim. In SkillOpt SearchQA, the final artifact improves independent held-out accuracy from 9/24 to 17/24, yet fails the predeclared stability threshold. It therefore supports local held-out benefit rather than stable transferable procedure learning.

Taken together, revised skill artifacts can produce retained behavioral effects, but a selected benchmark pass alone does not identify such an effect, and the effects established so far remain narrow rather than broadly generalizable.

## Attributing Observed Skill Gains

### Lifecycle Interventions

The experiments in this section intervene at different lifecycle stages. They do not estimate one common effect size. Each intervention changes one object while holding its relevant

**用于构建技能的反馈有助于解释其内容。** 这两种方法在将经验转化为修订方面的差异不同。SkillEvolver比较候选技能的成功和失败部署，然后修补缺失或误导性的指导。其独立的审计还会检查对训练实例常量的依赖（Zhang 等人，2026a）。这些机制直接针对遗漏的条件和实例特定的假设。我们调度和信号检测技能中保留的规则与这种侧重是一致的：它们描述了在重用程序时需要检查的内容。

CoEvoSkills 从协同进化的代理验证器处获取详细的修订反馈；而真实值预言机仅返回一个不透明的通过/失败信号（Zhang 等人 2026b）。因此，发现一个遗漏的条件取决于代理从任务和观察到的输出来重建它。这为一种涵盖许多可行性检查但遗漏了成功重用所需的条件的技能提供了一个合理的解释，就像调度示例中那样。两种方法都生成可重用的技能包；相关区别在于哪个失败信息成为持久的指导。我们的理解是，反馈覆盖范围和从源实例的抽象有助于确定结果技能的范围。

**将效用与进化过程连接起来。** 研究结果识别出有用的技能及其适用性差异；示例表明保留的指导与这些差异的关系。下一步研究将测试改进进入生命周期的位置，使用父节点到最终节点的比较以及对接收反馈、选择、技能内容和验证在SkillRevise、SkillHone和SkillOpt中的干预。

## 识别保留的技能效果

### 评估设计与对比

我们分析了三种具有不同修订机制的持久技能系统。SkillRevise将执行轨迹和诊断转化为有界的修复原则（Liu等人，2026b）。SkillHone积累练习历史并迭代地反思持久技能（Li和Hu，2026）。SkillOpt生成结构化的有界编辑，并在采用前使用验证门和拒绝缓冲区（Yang等人，2026）。这些系统作为代表性机制被研究，而不是作为可直接比较的排行榜条目。

在论文层面，证据具有探索性：单个干预使用冻结的输入和预声明的门，但综合研究旨在识别候选机制，而非估计基准范围内的普遍性。一个针对24项任务的SkillsBench面板测试历史结果是否能在新的部署中存活。该面板按比例分层，以暴露稳定性失败和方法分歧；它不是随机样本。随后三个受控机制案例比较在预声明变化下完整父级和最终产物。在每个对比中，我们保持执行者模型、测试框架、任务实例、预算和验证器版本固定，除非它是命名干预；精确提示、哈希值、种子和停止决策都保存在产物包中。任务是主要分析单元，而重复运行是相同任务条件对的部署。

必须保持四个比较的独立性。历史到新鲜重复的比较测试选定结果的保留，而不是相对于基线的优势。最终到无技能的比较估计新鲜效用，而不是修订的效果。只有实际父代到最终的比较才能识别修订效果。将此比较应用于受控变体测试有界迁移，而不是种群平均收益。

### 新鲜部署下的保留

此实验测试历史上选定的结果在最终工件冻结并重新部署时是否存活。保持任务、技能和条件固定并不能使执行确定性。在24任务面板中，12个任务在至少一个条件下在其两次重复之间改变了成功状态。聚合新鲜成功率为无技能的14/48，SkillRevise最终工件为16/48，SkillHone最终工件为12/48。两个匹配提升间隔都穿过零（表3）。

这些结果表明在常规报告实践中存在识别失败：缓存的或选定的成功不能被视为稳定的部署能力。它们并未证明任何修订方法都无效。该小组是故意的，并且缺乏每种方法的真实父条件。其恰当的结论更狭窄：在这个样本中，无论是最终版本与无技能比较，都无法建立明显的平均新鲜部署优势，在选定分数被称作保留之前，新鲜重复是必要的。

### 受控变化下的修订效果

这些机制测试在预先声明的任务变化下，实际的父到最终修订效果是否持续存在。在发票中，完整的最终技能修复了所有12个匹配的部署，并保留了从三个保持合同同时改变文档的独立语料库中抽取的九个部署的效果。这是稳定、同合同修订效果的强有力证据，但不是跨合同泛化的证据。

能源案例提供了较弱但方向上一致的证据：完整的最终工件改进了原始设置，并且在六次部署中的五次提高了5%的参数变体，而父项在六次部署中只有一次。样本太小，无法支持普遍性或泛化声明。在SkillOpt SearchQA中，最终工件独立保留的准确率从9/24提高到17/24，但未能通过预先声明的稳定性阈值。因此它支持局部保留收益，而不是稳定的可迁移程序学习。

综合来看，修订后的技能工件能够产生持续的行为效应，但仅通过选定的基准测试并不能识别出这种效应，且迄今为止建立起来的效应范围仍然较窄，而非具有广泛普适性。

## 归因于观察到的技能提升

### 生命周期干预

本节中的实验在不同生命周期阶段进行干预。它们不估计一个共同的效应量。每次干预改变一个对象，同时保持其相关

| Panel and contrast | Main result | Supported inference and boundary |
|---|---|---|
| **Panel A: Retention under fresh deployment** | | |
| Historical pass → fresh repeats | 12/24 tasks flip within at least one frozen condition; selected-outcome retention is 47.1% for SkillRevise and 55.0% for SkillHone | A selected pass is not a deployment probability; denominators differ and are not a method ranking. |
| SkillRevise final → no-skill | 16/48 vs. 14/48; $+4.2$ pp, 95% CI $[-10.4, 20.8]$ | No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate. |
| SkillHone final → no-skill | 12/48 vs. 14/48; $-4.2$ pp, 95% CI $[-20.8, 14.6]$ | No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate. |
| **Panel B: Revision effects under controlled variations** | | |
| Invoice parent → final | $0/12 \rightarrow 12/12$; three new same-contract corpora: $0/9 \rightarrow 9/9$ | Stable but narrow same-contract transfer. |
| Energy parent → final | Original task plus 5% parameter variant: $1/6 \rightarrow 5/6$ | Directional parameter transfer in a small stochastic case. |
| SkillOpt SearchQA parent → final | Independent held-out $9/24 \rightarrow 17/24$; stable-transfer gate fails | Local held-out gain, not stable general procedure learning. |

Table 3: Evidence for retained skill effects. Panel A contains 24 targeted tasks, three conditions, and two fresh deployments per condition (144/144 behavior-valid runs). Different historical-retention denominators preclude horizontal method ranking. The no-skill condition calibrates execution and selection variability; final-versus-no-skill does not identify revision effects.

lineage fixed: feedback-view studies hold the parent and failures fixed; optimizer-repeat studies hold the input evidence fixed; gate audits preserve accepted and rejected candidates; artifact surgery holds the surrounding artifact fixed; and verifier counterfactuals hold the output fixed.

Figure 2 makes the current coverage and its gaps explicit. The shared audit logic is that feedback access, candidate revision, validation acceptance, adoption into the persistent artifact, and retained behavioral improvement are distinct events.

## Revision and Selection Pathways

These interventions test whether the observed endpoint depends on the feedback view, optimizer sample, or selection gate. In SkillRevise, the complete three-revision pipeline succeeds on 53/86 tasks, compared with 28/86 when diagnosis is removed. Task-level discordance is 26 gains versus one loss, and 2,326 of 3,648 revision entries explicitly use verifier evidence. This demonstrates that diagnosis changes the selected endpoint and that the revision process is strongly evaluator-facing. It does not, by itself, establish which changes are retained under fresh deployment.

SkillHone separates access to feedback from stable benefit. Detailed sanitized, binary, and redacted-tail feedback obtain first-deployment success on 4/12, 4/12, and 5/12 cases, while four of 18 repeated cells flip. Feedback can trigger revisions, but these data do not support a stable advantage for one feedback view.

SkillOpt further separates candidate production from attribution and adoption. In matched SearchQA optimizer repeats, raw-gold and outcome-only views each produce four applied edits. Yet the preregistered between-condition effect size is only $D = .044 < .20$, despite weak lexical decodability, so a single patch cannot be cleanly attributed to feedback view. Across outcome-only and trajectory-only branches, all 6/6 candidates change and 3/6 pass the gate, but every acceptance is driven by the same Blackwater validation item. In OfficeQA, a selected one-item gain disappears in fresh matched deployment. Thus edit production, gate acceptance, and retained improvement cannot be collapsed into one event.

| Case | Result | Maximum claim |
|---|---|---|
| Invoice | $P = 0/12$, $P + B = 12/12$, $F - B = 0/12$, $F = 12/12$ | $B$ is necessary and sufficient for the narrow same-contract effect. |
| Energy | Local mediation in the original task; necessity and sufficiency do not survive the variant | Local component effect, not transferable mediation. |
| SearchQA | 9 repairs, 0 regressions; stable success 7/12, below the strong gate | Directional mediation on part of the held-out set. |

Table 4: Artifact-level mediation evidence.

## Artifact-Level Mediation

This experiment tests whether a feedback-derived component, rather than unrelated edits in the full artifact, causes the behavior after adoption. Let $P$ denote the complete parent, $F$ the complete final artifact, $P + B$ the parent with component $B$, and $F - B$ the final artifact with that component removed.

| 面板与对比 | 主要结果 | 支持推理与边界 |
|---|---|---|
| **面板A：新部署下的留存率** | | |
| 历史通过 → 新重复 | 12/24任务至少翻转一种冻结条件；选择结果保留是SkillRevise为47.1%，SkillHone为55.0% | 一个选择通过不是部署概率；分母不同，也不是方法排名。 |
| 技能修订最终 → 无技能 | SkillHone为16/48 vs. 14/48；+4.2pp，95% CI $[-10.4, 20.8]$ | 在此目标面板上没有明显的全新效用优势；不是从父级到最终修订的估计。 |
| 技能精炼最终 →无技能 | 12/48 与 14/48；−4.2 pp，95% CI $[-20.8, 14.6]$ | 在此靶向面板上，没有明显的全新效用优势；不是从父版到最终修订的估计。 |
| **面板B：在受控变化下的修订效果** | | |
| 发票父 →最终 | $0/12 \rightarrow 12/12$; 三个新的相同合约语料库：$0/9 \rightarrow 9/9$ | 稳定但狭窄的相同合约迁移。 |
| 能量父 →最终 | 原始任务加5%参数变体: $1/6 \rightarrow 5/6$ | 小随机情况下的方向参数迁移。 |
| SkillOpt SearchQA 父 → 最终 | 独立保留集 $9/24 \rightarrow 17/24$; 稳定迁移门控失败 | 本地保留集增益，不稳定泛化程序学习。 |

表3：保留技能效应的证据。面板A包含24个目标任务、三种条件和每个条件下的两次新鲜部署（144/144行为有效运行）。不同的历史保留分母妨碍了横向方法排名。无技能条件校准执行和选择变异性；最终与无技能对比无法识别修订效应。

谱系固定：反馈-视图研究固定父代和失败；优化器-重复研究固定输入证据；门控审计保留接受和拒绝的候选；工件手术固定周围工件；验证器反事实研究固定输出。

图2明确显示了当前覆盖范围及其差距。共享的审计逻辑是：反馈访问、候选修订、验证接受、采纳到持久化工件以及保留的行为改进是不同的事件。

## 修订与选择路径

这些干预措施检验观察到的终点是否取决于反馈视图、优化器样本或选择门。在SkillRevise中，完整的三次修订管道在53/86个任务上成功，而移除诊断时为28/86。任务级不一致性为26个收益与1个损失，且3,648个修订条目中有2,326个明确使用验证器证据。这表明诊断改变了选定的终点，且修订过程强烈依赖评估者。它本身并不能确定哪些变化在新鲜部署下被保留。

技能提升将获取反馈与稳定收益分离。详细清理、二进制及脱敏尾部反馈在4月12日、4月12日和5月12日的案例中取得首次部署成功，而18个重复单元格中有4个发生翻转。反馈可触发修订，但这些数据不支持任何一种反馈视图具有稳定优势。

技能优化器进一步将候选生成与归因和采纳分开。在匹配的SearchQA优化器重复中，原始金标和仅结果视图各产生四个应用编辑。但预先注册的组间效应量仅为 $D = .044 < .20$，尽管弱词汇解码性较差，因此单个补丁无法清晰地归因于反馈视图。在仅结果和仅轨迹分支中，所有6/6候选都发生变化，3/6通过门控，但每次接受都是由同一个Blackwater验证项驱动。在OfficeQA中，一个被选中的单项增益在新的匹配部署中消失。因此编辑生成、门控接受和保留改进不能合并为一个事件。

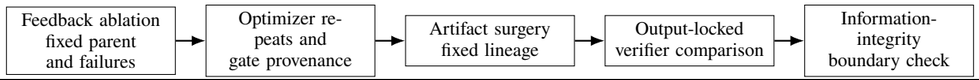
| Case | 结果 | 最大索赔 |
|---|---|---|
| 发票 | $P = 0/12$, $P + B = 12/12$, $F - B = 0/12$, $F = 12/12$ | $B$ 是必要且充分的。对于狭义相同合同效应。 |
| 能量 | 本地调解在原始任务；必要性和充分性无法存活变异体 | 局部组件效应，不可转移的媒介。 |
| SearchQA | 9个修复，0回归错误；稳定成功7/12，低于强门 | 部分的方向性中介保留集。 |

表4：工件级中介证据。

在仅结果和仅轨迹分支中，所有6/6候选都发生变化，3/6通过门控，但每次接受都是由同一个Blackwater验证项驱动。在OfficeQA中，一个被选中的单项增益在新的匹配部署中消失。因此编辑生成、门控接受和保留改进不能合并为一个事件。

## 工件级中介

该实验测试的是，在采用后导致行为的是由反馈派生的组件，而不是完整工件中无关的修改。令 $P$ 表示完整的父代，$F$ 表示完整的最终工件，$P + B$ 表示包含组件 $B$ 的父代，以及 $F - B$ 表示移除该组件的最终工件。




| Lifecycle stage | SkillRevise | SkillHone | SkillOpt |
|---|---|---|---|
| Feedback and revision | Diagnosis ablation; revision traces | Feedback-view ablation | Feedback branches; optimizer repeats |
| Selection and adoption | Revision history | Iterative honing history | Native validation gate; accepted and rejected candidates |
| Artifact mediation | Not yet matched at the same strength | Invoice and Energy surgery | SearchQA component surgery |
| Measurement | No output-locked comparison yet | Radar verifier swap | Spreadsheet dual scoring |
| Information integrity | Artifact forensic | Artifact forensic | Wrong-metadata and oracle controls |

Figure 2: Lifecycle attribution design and current evidence coverage. The upper panel locates each intervention; the lower panel records method coverage. Populated cells do not imply equivalent strength, and missing cells bound cross-method claims.

These interventions establish that an adopted artifact component can cause behavior. They do not show that the same mechanism is prevalent across methods. In particular, Skill-Revise lacks a matched surgery of equal strength, and the current cases should be read as existence and boundary evidence.

### Measurement Dependence under Fixed Outputs

This counterfactual tests whether a verdict changes when the output is fixed and only the measurement changes. In the Radar family, all 12 reconstructed outputs reproduce their historical original-verifier outcomes. Of six original passes, only one remains a pass under a reference-calibrated strict verifier; the five valid flips arise from specification-implied checks. The same behavior can therefore receive different valid verdicts because the measurement surfaces differ.

Measurement dependence can also operate in the other direction. For identical Spreadsheet workbook outputs, the reported value-based evaluator scores the complete SkillOpt artifact at 10/12, whereas joint value-and-formula recalculation scores it at 8/12. Representation, cache materialization, or formula inspection can therefore change what the evaluator observes. This dual scoring is not identical in design to the Radar output-locked swap, but it reinforces the narrower point that score is a property of output plus measurement, not behavior alone.

Neither result proves semantic failure, exploitation, or benchmark-wide prevalence. To identify evolution-induced verifier dependence, one would additionally need matched no-skill, parent, and final outputs under both original and calibrated verifiers, and test whether the parent-to-final verifier gap increases.

### Information-Integrity Boundary

This supporting check tests whether restricted answer or oracle information enters and controls the artifact. High-confidence direct-answer or oracle matches are absent from 86 SkillRevise and 85 SkillHone natural artifacts. SkillOpt conditions containing the feedback-derived answer-surface section follow incorrect metadata in 0/15 probes. In contrast, an unsafe hidden-oracle positive control changes probe success from 0/5 to 5/5 and leaves private-oracle or gold markers.

These checks show that the audit can detect direct exposure when it is deliberately introduced, while the natural artifacts do not support direct copying as a universal explanation. They do not prove the absence of implicit answer-following policies or all forms of leakage.

Observed score gains can arise at multiple lifecycle stages: feedback shapes candidate revisions, selection can amplify narrow improvements, adopted artifact components can mediate real behavior, and valid verifiers can disagree on fixed outputs. These mechanisms must be identified separately before a score gain is interpreted as skill learning.

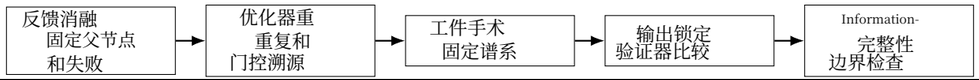
| Audit step | Required test → maximum supported claim |
|---|---|
| Validity | Execution and verifier status plus assertion coverage → eligible observation |
| Retention | Frozen actual-parent/final fresh repeats → retained artifact effect |
| Behavioral scope | Entity, parameter, input, contract, or family variants → bounded controlled transfer |
| Evolution pathway | Feedback ablation, optimizer repeats, and gate provenance → pathway dependence or selection mediation |
| Measurement | Output-locked calibrated verifier comparison → measurement dependence |
| Information integrity | Provenance, wrong-metadata, and positive control → exploitation or confirmed leakage, when supported |

Table 5: Evidence requirements and claim boundaries. The rows answer complementary questions rather than forming a scalar risk ladder.

## From Reported Gains to Auditable Claims

### Evidence Requirements and Claim Boundaries

The audit is not a scalar ladder from *safe* to *unsafe*. Its steps answer independent questions and impose distinct claim boundaries.




| 生命周期阶段 | SkillRevise | SkillHone | SkillOpt |
|---|---|---|---|
| 反馈与修订 | 诊断消融；修订轨迹 | 反馈视图消融 | 反馈分支；优化器重复 |
| 选择与采纳 | 修订历史 | 迭代打磨历史 | 本地验证关卡；已接受和被拒绝的候选者 |
| 文物调解 | 尚未在相同力量 | 发票和能源手术 | SearchQA组件手术 |
| 测量 | 尚未有输出锁定比较 | 雷达验证器交换 | 电子表格双重评分 |
| 信息完整性 | 文物取证 | 文物取证 | 错误元数据和预言机控制 |

图2：生命周期归因设计及当前证据覆盖范围。上层面板定位每个干预措施；下层面板记录方法覆盖范围。填充的单元格不表示同等强度，缺失的单元格界定跨方法主张。

这些干预措施表明，一个采用的工件组件可以导致行为。它们并未证明相同的机制在所有方法中都普遍存在。特别是 Skill-Revise 缺乏同等强度的匹配手术，当前案例应被视为存在性和边界证据。

### 固定输出下的测量依赖性

这个反事实测试在输出固定而仅测量变化时，判定是否会改变。在Radar系列中，所有12个重建输出都重现了其历史原始验证器结果。在六个原始通过中，只有一个在参考校准的严格验证器下仍然通过；五个有效翻转源于规范隐含的检查。因此，相同的行为可能因为测量表面不同而获得不同的有效判定。

测量依赖性也可以反向运作。对于相同的电子表格工作簿输出，基于值的评估器将SkillOpt完整工件评为10/12，而联合值和公式重新计算将其评为8/12。因此，表示、缓存材料化或公式检查可以改变评估器所观察到的内容。这种双重评分在设计中并不等同于Radar输出锁定的交换，但它加强了更狭窄的观点：分数是输出加测量的属性，而不仅仅是行为。

这两个结果都无法证明语义失败、利用或基准的普遍性。要识别由进化引起的验证器依赖性，还需要在原始和校准验证器下匹配无技能、父代和最终输出，并测试父代到最终的验证器差距是否增加。

### 信息完整性边界

此辅助检查测试是否受限答案或预言机信息会进入并控制该物品。在86个SkillRevise和85个SkillHone自然物品中，高置信度的直接答案或预言机匹配不存在。包含反馈衍生答案表面的SkillOpt条件

| 审计步骤 | 必需测试 → 最大支持声明 |
|---|---|
| 有效性 | 执行和验证器状态加上断言覆盖率 → 符合资格 |
| 保留 | 观察冻结实际父节点/最终新鲜重复 → 保留的工件效果 |
| 行为范围 | 实体、参数、输入、合约或家族变体 → 有界受控迁移 |
| 进化路径 | 反馈消融、优化器重复以及门控溯源 → 路径依赖或选择介导 |
| 测量 | 输出锁定校准验证器比较 →测量依赖 |
| 信息完整性 | 来源、错误元数据和正向控制 → 利用或确认泄露，当受支持时 |

表5：证据需求与主张边界。各行回答的是互补性问题，而非构成一个标量风险阶梯。

部分遵循错误的元数据，在0/15个探测中。相比之下，一个不安全的hidden-oracle阳性对照将探测成功率从0/5变为5/5，并保留private-oracle或金标记。

这些检查表明，审计能够检测到故意引入的直接暴露，而自然产生的伪影并不支持将直接复制作为普遍解释。它们并不能证明隐式答案跟随策略或所有形式的泄露不存在。

观察到的分数增益可能出现在多个生命周期阶段：反馈会塑造候选修订，选择可以放大狭窄的改进，采用的组件可以中介实际行为，而有效的验证器可能对固定输出存在分歧。在将分数增益解释为技能学习之前，必须分别识别这些机制。

## 从报告增益到可审计主张证据需求与主张边界

审计并非从安全到不安全的标量阶梯。它的步骤回答独立问题，并施加不同的主张边界。

A selected gain is weaker than a retained artifact effect; a retained effect on one instance is weaker than controlled transfer. Evolution-pathway and measurement claims remain meaningful even when retention is not established. Conversely, verifier-aware language or evaluator dependence must not be relabeled as leakage without restricted-information provenance and behavioral evidence.

### Lifecycle Reporting Protocol

An auditable self-evolution report should preserve the following minimum evidence:

- **Before optimization:** parent artifact, task and split identity, failure pool, environment, and verifier version.
- **During revision:** optimizer-visible traces and feedback, candidate patches, optimizer samples, prompts, and usage.
- **At selection and adoption:** accepted and rejected artifacts, gate items, per-item decisions, and exact adopted artifact.
- **At deployment:** frozen artifact, actual-parent and no-skill conditions, fresh repeats, controlled variants, and output bytes.
- **At scoring:** verifier logs, assertion coverage, reference calibration, and any alternative verifier applied to the same output.
- **For cost:** separate optimizer, executor, verifier, and invalid-attempt ledgers.

The complete artifact appendix should additionally record hashes, commands, seeds, environment revisions, timeouts, and stopping decisions. This protocol turns a post-evolution score into an auditable lineage and limits the paper's language to the strongest matched contrast actually performed.

## Related Work

### Skill Evolution and Empirical Evaluation

Agent-skill research spans generation, revision, selection, retrieval, organization, and reinforcement learning. SkillsBench and SWE-Skills-Bench compare skill and no-skill utility across diverse or software-engineering tasks (Li et al. 2026; Han et al. 2026). SkillLearnBench evaluates continual skill generation (Zhong et al. 2026), while SkillRevise, SkillHone, SkillOpt, SkillRL, and CoEvoSkills propose different revision or learning loops (Liu et al. 2026b; Li and Hu 2026; Yang et al. 2026; Xia et al. 2026; Zhang et al. 2026b). Recent empirical work broadens the object of study: From Raw Experience to Skill Consumption separates experience generation, extraction, and consumption (Huang et al. 2026); How Well Do Agentic Skills Work in the Wild studies realistic skill retrieval (Liu et al. 2026a); SkillCoach evaluates process-level skill use (Zhu et al. 2026); and SkillJuror isolates the effect of skill organization (Chen et al. 2026).

These studies show that outcome-only evaluation is already recognized as incomplete. Our focus is narrower and complementary: when a persistent skill is revised and the post-revision score rises, what evidence identifies a retained artifact effect rather than a selected or measurement-dependent endpoint? We therefore organize evidence by lifecycle contrast rather than by method score.

### Evaluation and Attribution of Agent Improvement

Agent evaluation increasingly examines process quality, sequential adaptation, verifier reliability, and counterfactual behavior. Counterfactual Trace Auditing measures how a provided skill changes an agent trajectory (Zhou et al. 2026a). SEA-Eval evaluates self-evolving agents beyond isolated episodes (Jiang et al. 2026). Co-evolving verifier work and grader audits study interactions between the policy and its measurement signal (Zhang et al. 2026b,c). More broadly, reward hacking and test-set contamination research demonstrates that performance on a proxy or exposed test need not imply the intended capability (Amodei et al. 2016; Oren et al. 2023).

We combine these concerns within the lifecycle of persistent skill revision. The central distinction is between retention, behavioral scope, evolution-and-selection pathway, and output-level measurement dependence. Information-integrity checks play a supporting role: they bound direct answer or oracle exposure, but do not convert every verifier-aware adaptation into a leakage claim.

## Limitations

Our evidence is mechanism-focused rather than a benchmark-wide prevalence study. The 24-task retention panel is purposive and contains only two fresh deployments per condition; it exposes instability but cannot estimate deployment rates. The controlled Invoice, Energy, SearchQA, Radar, Spreadsheet, and OfficeQA probes use small, heterogeneous task families rather than a fully crossed method-by-intervention design. SkillRevise, for example, lacks matched artifact surgery and output-locked measurement evidence of the same strength, while the strongest verifier swap is concentrated in the Radar family. We therefore do not claim that any observed mechanism is universal or that one method outperforms another.

Several conclusions are also bounded by observability. Output-locked comparisons identify measurement dependence but do not determine which verifier best captures intended behavior without an external specification. Artifact forensics and wrong-metadata controls can rule out simple direct exposure, but not implicit answer-following policies or every form of contamination. Finally, our scope is frozen-agent, feedback-driven systems with persistent external artifacts; parameter-updating self-evolution, multimodal skills, and online environments may exhibit different lifecycle dynamics. Confirmatory evidence will require a shared sampling plan that fully crosses actual parent and final artifacts, fresh repeats, controlled variants, and calibrated verifiers.

## Conclusion

We diagnose where the gains from self-evolving persistent skills reside and how far they extend. All three methods in the standard-task evaluations have positive paired gains in both

一个选定的收益弱于一个保留的实例效果；一个实例上的保留效果弱于受控迁移。即使保留未建立，进化路径和测量主张仍然有意义。相反，验证器感知语言或评估器依赖不得在没有受限信息来源和行为证据的情况下被重新标记为泄漏。

### 生命周期报告协议

一份可审计的自进化报告应保留以下最低证据：

- **优化前：** 父级工件、任务和分割身份、失败池、环境和验证器版本。
- **修订期间：** 优化器可见的轨迹和反馈、候选补丁、优化器样本、提示和使用情况。
- **在选择和采纳时：** 接受的和拒绝的工件、门控项、单项决策和精确采纳的工件。
- **在部署时：** 冻结的工件、实际父级和技能无效果条件、新鲜重复、受控变体和输出字节。
- **评分：** 验证器日志、断言覆盖率、参考校准，以及应用于相同输出的任何替代验证器。
- **对于成本：** 分离优化器、执行器、验证器和无效尝试账本。

完整的工件附录还应记录哈希值、命令、种子、环境修订、超时和停止决策。此协议将一个后进化分数转化为可审计的谱系，并将论文的语言限制为实际执行的最强匹配对比。

## 相关工作

### 技能进化和实证评估

代理技能研究涵盖生成、修订、选择、检索、组织和强化学习。技能基准和软件工程技能基准比较技能和无技能在不同或软件工程任务中的效用（Li 等人，2026；Han 等人，2026）。SkillLearnBench评估持续技能生成（Zhong 等人，2026），而SkillRevise、SkillHone、SkillOpt、SkillRL和CoEvoSkills提出不同的修订或学习循环（Liu 等人，2026b；Li 和 Hu，2026；Yang 等人，2026；Xia 等人，2026；Zhang 等人，2026b）。最近的实证工作扩展了研究对象：从原始经验到技能消费将经验生成、提取和消费分开（Huang 等人，2026）；《代理技能在野外效果如何》研究现实技能检索（Liu 等人，2026a）；SkillCoach评估过程级技能使用（Zhu 等人，2026）；SkillJuror隔离技能组织的效果（Chen 等人，2026）。

这些研究表明，仅基于结果的评估已被公认为是不完整的。我们的关注点更狭窄且具有互补性：当一项持久的技能被修订时，修订后的分数上升，哪些证据能识别出保留的遗留效应，而不是被选择或测量依赖的终点？因此，我们按生命周期对比来组织证据，而不是按方法分数来组织。

### 代理改进的评估与归因

代理评估越来越多地考察过程质量、序列适应、验证器可靠性和反事实行为。反事实轨迹审计测量提供的技能如何改变代理轨迹（Zhou 等人 2026a）。SEA-Eval 超越孤立场景评估自进化代理（Jiang 等人 2026）。协同进化的验证器工作和评分器审计研究策略与其测量信号之间的交互（Zhang 等人 2026b,c）。更广泛地说，奖励破解和测试集污染研究证明，在代理或暴露的测试上的表现不一定意味着预期的能力（Amodei 等人 2016；Oren 等人 2023）。

我们将这些关注点结合在持久技能修订的生命周期中。核心区别在于保持、行为范围、进化与选择路径以及输出级测量依赖。信息完整性检查起辅助作用：它们限制直接答案或预言机暴露，但不会将每个验证器感知的适应都转化为泄露声明。

## 局限性

我们的证据是机制导向的，而非跨基准的普遍性研究。24任务持久性面板是刻意设计的，每个条件下仅包含两次新鲜部署；它暴露了不稳定性，但无法估计部署率。受控的Invoice、Energy、SearchQA、Radar、Spreadsheet和OfficeQA探测任务使用小型异构任务族，而非完全交叉的方法-干预设计。例如，SkillRevise缺乏同等强度的匹配工件手术和输出锁定测量证据，而最强的验证器交换集中在Radar家族。因此，我们不声称任何观察到的机制是普遍的，也不声称一种方法优于另一种方法。

一些结论也受限于可观察性。输出锁定比较能识别测量依赖性，但无法确定哪个验证器能最好地捕捉预期行为，除非有外部规范。工件取证和错误元数据控制可以排除简单的直接暴露，但无法排除隐式答案跟随策略或任何形式的污染。最后，我们的范围是冻结代理、反馈驱动的系统，具有持久的 外部工件；参数更新的自进化、多模态技能和在线环境可能表现出不同的生命周期动态。确认性证据将需要一个共享的采样计划，该计划完全交叉实际父代和最终工件、新鲜重复、受控变体和校准验证器。

## 结论

我们诊断了自我进化持久的技能带来的收益及其范围。在标准任务评估中的三种方法在两者中都有正向配对收益。

runs, but task variation separates their utility profiles. SkillEvolver remains positive across representation, instance, and requirement changes; SPARK combines gains on two probes with a decline on instance changes; CoEvoSkills declines on all three. The inspected traces identify decision criteria and correctness checks expressed during successful reuse. Complementary lifecycle interventions show that adopted artifact components can change behavior, selected outcomes can disappear under fresh deployment, and fixed outputs can receive different verdicts under different verifiers. Together, these findings distinguish useful artifacts, their behavioral scope, and the processes that produce or measure their apparent improvement.

A higher benchmark score should therefore be reported only at the strongest level supported by matched evidence: a selected gain, a retained artifact effect, bounded controlled transfer, evolution-pathway dependence, or measurement dependence. We provide a lifecycle audit that links each claim to the required intervention and reporting evidence. Claims of transferable skill learning require frozen parent-to-final improvement that survives fresh deployment and predeclared task variation under valid measurement; weaker evidence should be reported more narrowly.

## References


Amodei, D.; Olah, C.; Steinhardt, J.; Christiano, P.; Schulman, J.; and Mané, D. 2016. Concrete Problems in AI Safety. arXiv:1606.06565.

Chen, Z.; Guo, Z.; Huang, B.; Lu, B.; Lin, J.; Zhou, Y.; and Zhang, W. 2026. SkillJuror: Measuring How Agent Skill Organization Changes Runtime Behavior. arXiv:2606.11543.

Han, T.; Zhang, Y.; Song, W.; Fang, C.; Chen, Z.; Sun, Y.; and Hu, L. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? arXiv:2603.15401.

Huang, Z.; Xu, J.; Yang, Y.; Gong, Z.; Yang, Q.; Tian, M.; Wang, X.; Lv, C.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, X.; Chen, D.; Zheng, X.; and Luo, C. 2026. From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills. arXiv:2605.23899.

Jiang, S.; Ma, L.; Hong, Z.; Wang, K.; Lu, Z.; Wang, T.; Chen, S.; Zhang, J.; Pan, T.; Li, W.; Liang, J.; and Xiao, Y. 2026. SEA-Eval: A Benchmark for Evaluating Self-Evolving Agents Beyond Episodic Assessment. arXiv:2604.08988.

Li, X.; Liu, Y.; Chen, W.; You, B.; Di, Z.; He, Y.; Zheng, S.; Choe, K. W.; Sun, J.; Wang, S.; Tao, C.; Li, B.; Zhao, X.; Geng, H.; Wu, X.; Zhou, J.; Chen, X.; Xing, H.; Li, Y.; Zeng, Q.; Wang, D.; Wang, Y.; Chaim, R. B.; Jiang, P.; Shen, H.; Kong, L.; Liu, X.; Wang, R.; Liu, X.; Li, J.; Lan, X.; Lin, Y.; Ye, W.; He, J.; Li, S.; Zhang, Y.; Gao, Y.; Li, Y.; Ma, Z.; Jing, L.; Wang, T.; Li, K.; Xue, Y.; Lyu, H.; He, Y.; Tian, Y.; Wu, S.; Wang, B.; Gao, Y.; Chen, B.; Liu, L.; Cheng, S.; Bao, J.; Tong, S.; Xu, S.; Zhuo, T. Y.; Ye, T.; Qi, Q.; Li, M.; Liao, L.; Tan, Z.; Shi, C.; Tang, X.; Tankasala, S.; Yuan, B.; Qian, Y.; Tu, J.; Wang, C.; Sun, Y.; Wang, W.; Taylor, A.; Yang, Z.; Guan, C.; Dong, Z.; Zhang, X.; Dillmann, S.; chung Lee, H.; and Song, D. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. arXiv:2602.12670.

Li, Z.; and Hu, Y. 2026. SkillHone: A Harness for Continual Agent Skill Evolution Through Persistent Decision History. arXiv:2606.08671.

Liu, Y.; Ji, J.; An, L.; Jaakkola, T.; Zhang, Y.; and Chang, S. 2026a. How Well Do Agentic Skills Work in the Wild: Benchmarking LLM Skill Usage in Realistic Settings. arXiv:2604.04323.

Liu, Y.; Su, Z.; Xie, L.; Zhang, Y.; Zong, Q.; Guo, J.; Xie, Z.; Ji, Y.; Yim, Y.; Luo, H.; Ren, X.; Ruan, C.; Li, H.; and Song, Y. 2026b. SkillRevise: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision. arXiv:2606.01139.

Oren, Y.; Meister, N.; Chatterji, N.; Ladhak, F.; and Hashimoto, T. B. 2023. Proving Test Set Contamination in Black Box Language Models. arXiv:2310.17623.

Xia, P.; Chen, J.; Wang, H.; Liu, J.; Zeng, K.; Wang, Y.; Han, S.; Zhou, Y.; Zhao, X.; Chen, H.; Zheng, Z.; Xie, C.; and Yao, H. 2026. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. arXiv:2602.08234.

Yang, Y.; Gong, Z.; Huang, W.; Yang, Q.; Zhou, Z.; Huang, Z.; Li, Y.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, Y.; Chen, D.; Yang, X.; and Luo, C. 2026. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. arXiv:2605.23904.

Zhang, G.; Zhu, E.; Zhou, J.; Jia, C.; and Wang, H. 2026a. SkillEvolver: Skill Learning as a Meta-Skill. arXiv:2605.10500.

Zhang, H.; Fan, S.; Zou, H. P.; Chen, Y.; Wang, Z.; Zhou, J.; Li, C.; Huang, W.-C.; Yao, Y.; Zheng, K.; Liu, X.; Li, X.; and Yu, P. S. 2026b. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. arXiv:2604.01687.

Zhang, X.; Wang, G.; Cui, Y.; Li, Z.; Qiu, W.; Zhu, B.; and He, P. 2026c. Who Grades the Grader? Co-Evolving Evaluation Metrics and Skills for Self-Improving LLM Agents. arXiv:2607.12790.

Zhong, S.; Lu, Y.; Ning, J.; Wan, Y.; Feng, L.; Ao, Y.; Ribeiro, L. F. R.; Dreyer, M.; Ammirati, S.; and Xiong, C. 2026. SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks. arXiv:2604.20087.

Zhou, X.; Liu, J.; Li, L.; Rossi, R. A.; and Hu, X. 2026a. Counterfactual Trace Auditing of LLM Agent Skills. arXiv:2605.11946.

Zhou, Y.; Dong, Z.; Wang, Z.; Jin, C.; Zhao, S.; Guo, B.; Gu, D.; Zhang, L.; Zhou, M.; and Metaxas, D. N. 2026b. Evidence Over Plans: Online Trajectory Verification for Skill Distillation. arXiv:2605.09192.

Zhu, J.; Mao, K.; Guo, Y.; He, D.; Xu, S.; Gu, S.; and Yue, Y. 2026. SkillCoach: Self-Evolving Rubrics for Evaluating and Enhancing Agentic Skill-Use. arXiv:2607.01874.


我们诊断了自演化持久技能的收益来自何处以及它们能延伸多远。在标准任务评估中的三种方法在所有运行中都具有正向配对收益，但任务变化将它们的效用特征分开了。SkillE-volver在表示、实例和需求变化中始终保持正值；SPARK在两个探针上结合收益，但在实例变化中收益下降；CoEvoSkills在所有三种情况下都下降。检查的轨迹识别了在成功重用时表达的决策标准和正确性检查。互补的生命周期干预表明，采用的工件组件可以改变行为，选定的结果在全新部署下可能会消失，固定的输出在不同验证器下可能会收到不同的裁决。这些发现共同区分了有用的工件、它们的行为范围以及产生或衡量它们明显改进的过程。

因此，只有在有匹配证据支持的最强级别下，才应报告更高的基准分数：一个选定的收益、一个保留的文物效应、有界的受控迁移、进化路径依赖性或测量依赖性。我们提供生命周期审计，将每个主张与所需的干预措施和报告证据联系起来。可迁移技能学习的主张需要冻结的从父级到最终改进，这种改进在新的部署和预声明的任务变化下仍然有效，并且符合有效的测量；较弱的证据应更严格地报告。

## 参考文献


Amodei, D.; Olah, C.; Steinhardt, J.; Christiano, P.; Schulman, J.; 和 Mané, D. 2016. AI安全的具体问题. arXiv:1606.06565. Chen, Z.; Guo, Z.; Huang, B.; Lu, B.; Lin, J.; Zhou, Y.; 和 Zhang, W. 2026. SkillJuror: 衡量代理技能组织如何改变运行时行为. arXiv:2606.11543. Han, T.; Zhang, Y.; Song, W.; Fang, C.; Chen, Z.; Sun, Y.; 和 Hu, L. 2026. SWE-Skills-Bench: 代理技能是否真的有助于实际软件工程? arXiv:2603.15401. Huang, Z.; Xu, J.; Yang, Y.; Gong, Z.; Yang, Q.; Tian, M.; Wang, X.; Lv, C.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, X.; Chen, D.; Zheng, X.; 和 Luo, C. 2026. 从原始经验到技能消耗: 对模型生成的代理技能的系统研究. arXiv:2605.23899. Jiang, S.; Ma, L.; Hong, Z.; Wang, K.; Lu, Z.; Wang, T.; Chen, S.; Zhang, J.; Pan, T.; Li, W.; Liang, J.; 和 Xiao, Y. 2026. SEA-Eval: 超越情景评估的自进化代理评估基准. arXiv:2604.08988. Li, X.; Liu, Y.; Chen, W.; You, B.; Di, Z.; He, Y.; Zheng, S.; Choe, K. W.; Sun, J.; Wang, S.; Tao, C.; Li, B.; Zhao, X.; Geng, H.; Wu, X.; Zhou, J.; Chen, X.; Xing, H.; Li, Y.; Zeng, Q.; Wang, D.; Wang, Y.; Chaim, R. B.; Jiang, P.; Shen, H.; Kong, L.; Liu, X.; Wang, R.; Liu, X.; Li, J.; Lan, X.; Lin, Y.; Ye, W.; He, J.; Li, S.; Zhang, Y.; Gao, Y.; Li, Y.; Ma, Z.; Jing, L.; Wang, T.; Li, K.; Xue, Y.; Lyu, H.; He, Y.; Tian, Y.; Wu, S.; Wang, B.; Gao, Y.; Chen, B.; Liu, L.; Cheng, S.; Bao, J.; Tong, S.; Xu, S.; Zhuo, T. Y.; Ye, T.; Qi, Q.; Li, M.; Liao, L.; Tan, Z.; Shi, C.; Tang, X.; Tankasala, S.; Yuan, B.; Qian, Y.; Tu, J.; Wang, C.; Sun, Y.; Wang, W.; Taylor, A.; Yang, Z.; Guan, C.; Dong, Z.; Zhang, X.; Dillmann, S.; chung Lee, H.; 和 Song, D. 2026. SkillsBench: 基准测试代理技能在不同任务中的表现. arXiv:2602.12670. Li, Z.; 和 Hu, Y. 2026. SkillHone: 通过持久决策历史持续进化代理技能的框架. arXiv:2606.08671. Liu, Y.; Ji, J.; An, L.; Jaakkola, T.; Zhang, Y.; 和 Chang, S. 2026a. 代理技能在现实中的表现如何: 基准测试真实场景中的LLM技能使用. arXiv:2604.04323. Liu, Y.; Su, Z.; Xie, L.; Zhang, Y.; Zong, Q.; Guo, J.; Xie, Z.; Ji, Y.; Yim, Y.; Luo, H.; Ren, X.; Ruan, C.; Li, H.; 和 Song, Y. 2026b. SkillRevise: 通过轨迹条件技能修订改进LLM生成的代理技能. arXiv:2606.01139. Oren, Y.; Meister, N.; Chatterji, N.; Ladhak, F.; 和 Hashimoto, T. B. 2023. 证明黑盒语言模型的测试集污染. arXiv:2310.17623. Xia, P.; Chen, J.; Wang, H.; Liu, J.; Zeng, K.; Wang, Y.; Han, S.; Zhou, Y.; Zhao, X.; Chen, H.; Zheng, Z.; Xie, C.; 和 Yao, H. 2026. SkillRL: 通过递归技能增强强化学习进化代理. arXiv:2602.08234. Yang, Y.; Gong, Z.; Huang, W.; Yang, Q.; Zhou, Z.; Huang, Z.; Li, Y.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, Y.; Chen, D.; Yang, X.; 和 Luo, C. 2026. SkillOpt: 自进化代理技能的执行策略. arXiv:2605.23904. Zhang, G.; Zhu, E.; Zhou, J.; Jia, C.; 和 Wang, H. 2026a. SkillEvolver: 技能学习作为一种元技能. arXiv:2605.10500. Zhang, H.; Fan, S.; Zou, H. P.; Chen, Y.; Wang, Z.; Zhou, J.; Li, C.; Huang, W.-C.; Yao, Y.; Zheng, K.; Liu, X.; Li, X.; 和 Yu, P. S. 2026b. CoEvoSkills: 通过协同进化验证自进化代理技能. arXiv:2604.01687. Zhang, X.; Wang, G.; Cui, Y.; Li, Z.; Qiu, W.; Zhu, B.; 和 He, P. 2026c. 谁评估评估者? 协同进化评估指标和技能以改进自提升LLM代理. arXiv:2607.12790. Zhong, S.; Lu, Y.; Ning, J.; Wan, Y.; Feng, L.; Ao, Y.; Ribeiro, L. F. R.; Dreyer, M.; Ammirati, S.; 和 Xiong, C. 2026. SkillLearnBench: 基准测试现实任务上代理技能生成的持续学习方法. arXiv:2604.20087. Zhou, X.; Liu, J.; Li, L.; Rossi, R. A.; 和 Hu, X. 2026a. LLM代理技能的逆事实轨迹审计. arXiv:2605.11946. Zhou, Y.; Dong, Z.; Wang, Z.; Jin, C.; Zhao, S.; Guo, B.; Gu, D.; Zhang, L.; Zhou, M.; 和 Metaxas, D. N. 2026b. 证据胜于计划: 技能蒸馏的在线轨迹验证. arXiv:2605.09192. Zhu, J.; Mao, K.; Guo, Y.; He, D.; Xu, S.; Gu, S.; 和 Yue, Y. 2026. SkillCoach: 自进化评估和增强代理技能使用的评分标准. arXiv:2607.01874.